EBERHARD KARLS UNIVERSITÄT TÜBINGEN

Department of Computer Science · Database Systems

SQL as a Programming Language

Implementing Distance-Vector Routing with Recursive CTEs in DuckDB

Mehdi Hoseyni

Seminar: SQL is a Programming Language · Summer Semester 2026

Outline

1. Motivation and Research Question
2. Distance-Vector Routing Protocol
3. The Bellman-Ford Equation
4. SQL Recursive CTEs: The Challenge
5. DuckDB `USING KEY`: The Solution
6. Standard vs Keyed CTE Comparison
7. Implementation: The Algorithmic Core
8. Convergence Results
9. Network Failure and Re-convergence
10. Live Demonstration
11. Benchmark Results
12. Key Findings and Conclusion

Presentation duration: 20 minutes + discussion
Demo: Interactive Streamlit application (localhost:8501)

Motivation and Research Question

Research Question

Can SQL serve as a **first-class programming language** for complex algorithmic simulation?

Approach: Implement the Distance-Vector Routing protocol entirely within SQL using DuckDB's recursive CTEs.

Goal: Show that declarative logic with state-aware recursion achieves global optimization through local updates.

Why It Matters

- **Traditional view:** SQL is only for data retrieval
- **Our claim:** Modern SQL extensions enable full simulation of distributed systems
- **Implication:** Database engines can function as protocol validation environments

Thesis: DuckDB's `USING KEY` provides a 1:1 conceptual mapping to physical router behavior.

Distance-Vector Routing Protocol

Core Idea — "Routing by Rumor":

- Each router maintains a *distance vector* of costs to all destinations
- Routers share tables **only with immediate neighbors**
- Each router updates its table if a cheaper path is discovered
- Process repeats until **convergence**

Algorithm Steps:

T=0: Each router knows only direct neighbor costs
T=1: Share tables; apply relaxation rule
T=2+: Repeat until convergence
T=k: Fixed point — all paths optimal

Key Properties

Distributed	No central controller
Iterative	Updates in rounds
Asynchronous	Independent timing
Convergent	≤ |V|−1 rounds

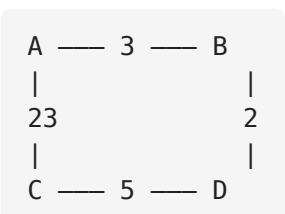
Real-World Protocols

- **RIP** (v1, v2) — max 15 hops
- **EIGRP** — Cisco hybrid protocol
- **BGP** — Internet backbone routing

The Bellman-Ford Equation

$$d(u, v) = \min_{w \in \text{Adj}(u)} \{\text{cost}(u, w) + d(w, v)\}$$

Example Network: 4 Routers



Direct link A→C costs 23, but the indirect path A→B→C costs only 3+2 = **5**.

Step-by-Step Convergence

Round	Discovery from Router A
T=0	B=3, C=23 (direct neighbors)
T=1	C=5 via B (3+2 < 23), D=28 via C
T=2	D=10 via B (3+2+5 < 28)
T=3	No changes — **converged**

SQL Recursive CTEs: The Challenge

The Problem

Standard SQL recursive CTEs are **append-only**:

- `UNION ALL` adds new rows each iteration
- Cannot update or delete existing rows
- All possible walks are enumerated
- Final `GROUP BY ... MIN(cost)` extracts answer

Result: Exponential row growth on dense graphs. SQLite exhausts memory beyond 12 nodes.

What DVR Needs

A physical router's forwarding table:

- **In-place updates:** overwrite when better route found
- **Bounded size:** only $O(V^2)$ entries at any time
- **Auto-termination:** stop when no improvement
- **Per-key pruning:** one entry per (src, dst)

Core mismatch: Standard recursion builds a growing set of paths. DVR requires a fixed-size table refined iteratively.

DuckDB `USING KEY`: The Solution

`WITH RECURSIVE ... USING KEY (from_node, to_node)` — DuckDB's extension that designates columns as a unique key. During recursion, new rows with matching keys **replace** old rows instead of appending.

In-Place Updates

Keying on `(from_node, to_node)` maintains one entry per router pair. Cheaper paths overwrite old entries.

Bounded Working Set

Working set stays at $O(V^2)$ regardless of graph density. No walk explosion.

Auto-Termination

Recursion halts when delta set is empty. No explicit iteration bound needed.

Working Set	Walk Explosion	Termination	Post-Hoc Agg.
$O(V^2)$	None	Automatic	Not needed

Standard CTE vs Keyed CTE

SQLITE: STANDARD CTE

```
WITH RECURSIVE bellman(s,t,cost,hop)
AS (
  SELECT from_node, to_node,
         cost, to_node
  FROM edges
  UNION ALL
  SELECT b.s, e.to_node,
         b.cost + e.cost, b.hop
  FROM bellman b
  JOIN edges e ON b.t = e.from_node
  WHERE b.s <> e.to_node
)
-- POST-HOC aggregation required
SELECT s, t, MIN(cost), hop
FROM bellman GROUP BY s, t
```

DUCKDB: KEYED CTE

```
WITH RECURSIVE routing(s,t,cost,hop)
    USING KEY (s, t) AS (
  SELECT from_node, to_node,
         cost, to_node
  FROM edges
  UNION
  SELECT r.s, e.to_node,
         MIN(r.cost + e.cost),
         arg_min(r.hop,
                 r.cost + e.cost)
  FROM routing r
  JOIN edges e ON r.t = e.from_node
  WHERE r.s <> e.to_node
  GROUP BY r.s, e.to_node
)
SELECT * FROM routing
```

Key difference: Standard CTE uses `UNION ALL` (accumulates all walks), requiring final `GROUP BY`. DuckDB uses `UNION` + `USING KEY` (prunes in-place), keeping only the best cost per (s, t).

The Algorithmic Core

```
WITH RECURSIVE routing_table(
  from_node, to_node,
  best_cost, next_hop)
  USING KEY (from_node, to_node) AS (
  -- T=0: Direct neighbor discovery
  SELECT from_node, to_node,
         cost, to_node AS next_hop
  FROM edges
  UNION
  -- Bellman-Ford relaxation
  SELECT r.from_node, e.to_node,
    MIN(r.best_cost + e.cost),
    arg_min(r.next_hop,
            r.best_cost + e.cost)
  FROM routing_table AS r
  JOIN edges AS e
    ON r.to_node = e.from_node
  LEFT JOIN recurring.routing_table
    AS ex ON ex.from_node = r.from_node
       AND ex.to_node = e.to_node
  WHERE r.from_node <> e.to_node
    AND (ex.best_cost IS NULL
      OR r.best_cost + e.cost
         < ex.best_cost)
  GROUP BY r.from_node, e.to_node
)
SELECT * FROM routing_table
ORDER BY from_node, to_node;
```

Line-by-Line

`USING KEY`: Stateful row updates

Base case: Direct neighbor costs

`JOIN edges`: Neighbor propagation

`recurring.*`: Accumulated FIB

`IS NULL OR <`: Relaxation condition

`MIN` + `arg_min`: Best advertisement

This single query replaces hundreds of lines of imperative routing code.

Convergence Results: Full Network

From	To	Cost	Next Hop
A	B	**3**	B
A	C	**5**	B
A	D	**10**	B
B	A	**3**	A
B	C	**2**	C
B	D	**7**	C
C	A	**5**	B
C	B	**2**	B
C	D	**5**	D
D	A	**10**	C
D	B	**7**	C
D	C	**5**	C

Observations

- **A → C = 5 (via B):** Indirect A-B-C (3+2) beats direct (23)
- **A → D = 10 (via B):** Multi-hop A-B-C-D
- **Convergence:** 3 rounds for 4 nodes
- All 12 pairs correctly computed

Insert screenshot:
Phase 2 — graph with shortest path overlay

Network Failure and Re-convergence

Simulating Link Failure

```
DELETE FROM edges
WHERE (from_node='B' AND to_node='C')
   OR (from_node='C' AND to_node='B');
```

Re-running the **same query** discovers next-best paths automatically.

Route	Before	After	Δ
A → C	5	**23**	+18
A → D	10	**28**	+18
B → C	2	**26**	+24
B → D	7	**31**	+24
C → D	5	5	0

Insert screenshot:
Phase 4 — before/after graph comparison + cost impact chart

INTERACTIVE DEMONSTRATION

Live Demo

Phase 1: Network Setup
topology + graph statistics

Phase 2: Convergence
routing table + path visualization

http://localhost:8501

Streamlit · DuckDB · Pyvis · Plotly

Benchmark Results

Insert screenshot:
Execution latency vs network size

Insert screenshot:
Log-log complexity analysis

Key Findings

- DuckDB: **5–15x speedup** over SQLite
- SQLite: **infeasible beyond 12 nodes**
- DuckDB: **polynomial scaling** O(V·E)
- Python: validated reference baseline

Backend	Complexity	Max N
DuckDB USING KEY	O(V·E)	50+
SQLite Standard	O(V·E·P)	11
Python Bellman-Ford	$O(V^2 \cdot E)$	50+
Python Dijkstra	O(V(V+E)logV)	50+
Python Floyd-Warshall	$O(V^3)$	50+

Key Findings

1. SQL as a PL

SQL can fully implement distributed routing protocols. A single recursive query replaces hundreds of lines of imperative code.

2. USING KEY Impact

Eliminates exponential path explosion. Working set bounded at $O(V^2)$. Achieves 5–15x speedup over SQLite.

3. Dynamic Resilience

Link failure via SQL `DELETE`; re-running the same query discovers next-best paths. No code changes needed.

Central result: Modern SQL is no longer “just for queries.” With state-aware recursion, it is a first-class language for simulating and analyzing complex distributed protocols.

Conclusion and Future Work

Conclusion

- DuckDB's `USING KEY` provides a **1:1 mapping** between SQL recursion and router behavior
- Built a complete **interactive evaluation suite** with 6 phases
- Keyed CTEs maintain **polynomial scaling**; standard CTEs degrade exponentially
- Declarative approach: **more concise** and **verifiably correct**

Future Research

- **Path-Vector (BGP):** SQL array types for AS-path tracking
- **Link-State (OSPF):** Dijkstra via recursive SQL
- **Dynamic Traffic:** Real-time metrics in edges table
- **Large-Scale:** Internet AS-graph convergence

References:
[1] R. Bellman, "On a Routing Problem," *Q. Appl. Math.*, 1958.
[2] DuckDB Documentation, "Recursive CTEs with USING KEY," 2026.
[3] A. S. Tanenbaum, *Computer Networks*, 6th ed., Pearson, 2021.

END OF PRESENTATION

Thank You

Questions and Discussion

Mehdi Hoseyni

Database Systems Seminar · University of Tübingen · Summer 2026

Backup Slides

Additional material for discussion

Complexity Comparison (Backup)

Algorithm	Time	Space	Distributed	Neg. Weights
Bellman-Ford	$O(V^2 \cdot E)$	$O(V^2)$	Yes	Yes
Dijkstra (heap)	$O(V(V+E)\log V)$	$O(V^2)$	No	No
Floyd-Warshall	$O(V^3)$	$O(V^2)$	No	Yes
DuckDB USING KEY	$O(V \cdot E)$	$O(V^2)$	N/A	No
SQLite Standard CTE	$O(V \cdot E \cdot P)$	$O(V^2 \cdot P)$	N/A	No

P = walk count: In SQLite, P denotes distinct walks enumerated before the final GROUP BY. For dense graphs, P grows exponentially with |V|.

CTE Semantic Comparison (Backup)

Aspect	Standard CTE (SQLite)	Keyed CTE (DuckDB)
Recursion model	Append-only (UNION ALL)	In-place update (UNION + KEY)
Working set	Grows each iteration	Bounded at $O(V^2)$
Row pruning	None during recursion	Auto per-key minimum
Aggregation	Post-hoc GROUP BY	Built into recursion
Termination	Explicit iteration bound	Automatic (empty delta)
Memory	$O(V^2 \cdot P)$, P = walks	$O(V^2)$
Walk explosion	High (exponential)	None (pruned by design)